0 9

SOL- — Detection-Pressure Scheduler

Field	Value
Revision	1
Last modified	2026-07-22T21:50:00Z
Rank	9
Closes	PC-7 (execution half — guards that exist but never run where the defect lives), the corpus-3 detection-pressure asymmetry
Seam	dispatch/schedule (feeds the autonomous loop's queue) — composing with SOL-02, which blocks the release on what this scheduler failed to freshen
POC	`poc/sol09_detection_pressure/` — GREEN 8/8, RED-first
Anchors mechanized (no new anchor)	§11.4.118 (discovery pressure), §11.4.189 (most-reopened first), §11.4.132(d)

. The measured problem

- Corpus 3's one genuine recurrence surfaced at **24 days — the next SWEEP, not the next use** (§11.4.118: we only see what we test). Its ~0.3% reopen figure is therefore an **evidence-bounded floor under weak detection pressure, not a proven ceiling** (corpus 3 §5.2a, UNKNOWN-1).
- Corpus 1: **two standing guards, finally executed during the forensics, immediately emitted the FAILs that had been latent the whole time** (one reproduced the operator's loudest defect 3/3) [PC-7]. Registration was treated as coverage; execution was nobody's job.

- The corpus-1-vs-corpus-3 rate split (52% vs ~0.3%) is explained by detection pressure + closure-evidence class, not machinery (corpus 3 §5.2) — pressure is a *causal input to the measured rate*, so leaving it to chance makes every quality number flattering and meaningless.
- Corpus 2's latency distribution (minutes-to-hours) shows what high pressure looks like: its users hit every escape immediately — painful, but *honest*.

. The mechanism

Detection latency becomes a scheduled, budgeted, risk-ordered quantity:

1. Every registered, topology-present guard carries an implicit freshness contract: a verdict **on the current artifact fingerprint**, no older than its staleness budget.
2. The scheduler emits the **re-run queue**: never-executed guards, stale-fingerprint guards, over-age guards — ordered **most-reopened-first** (§11.4.189: the empirically-most-fragile set gets the deepest scrutiny first), then stalest-first.
3. The autonomous loop consumes the queue as standing work (§11.4.94 zero-idle has a concrete feed); SOL-02 remains the blocking backstop at release for whatever the schedule missed.
4. Topology-absent guards are enumerated, never queued (no false pressure — §11.4.201(1)); an empty registry is `BLIND`, never "all fresh".

. POC results

RED → GREEN **8/8**: fresh guard not queued; never-executed / stale-fingerprint / over-age all queued; the reopens=4 guard ordered first; topology-absent enumerated not queued; empty registry blind.

. The failure this makes IMPOSSIBLE

A registered guard can no longer sit un-executed indefinitely while counting as coverage in anyone's head: it is either fresh-on-the-current-artifact, or in a visible, ordered work queue that the zero-idle loop drains. The corpus-3 24-day latency becomes a chosen budget number, not an accident of when someone looked.

. What it still does NOT catch (honest boundary)

1. **Pressure creates opportunity, not oracle quality** — re-running a weak guard often proves nothing (R1.2); oracle strength stays with SOL-04 + §1.1 mutations. SQLite's lesson (R4.1) is carried explicitly: even perfect re-execution of one lens misses the complement only an *orthogonal* lens sees — fuzz/chaos-class discovery (§11.4.85) is a different feed this scheduler does not generate.
2. **Budgets are consumer data** and must respect the trust economy (R6.3: past the calibration point, added interruptive load *reduces* total enforcement — the queue is work-feed, not a blocking alarm fleet; only SOL-02 blocks, and only at release).
3. **Fingerprint freshness assumes fingerprints change when the artifact does** — a consumer whose fingerprint is coarse (e.g. version string, not content hash) under-detects staleness; read-from-target content fingerprints (§11.4.115(F)) are the integration requirement.